

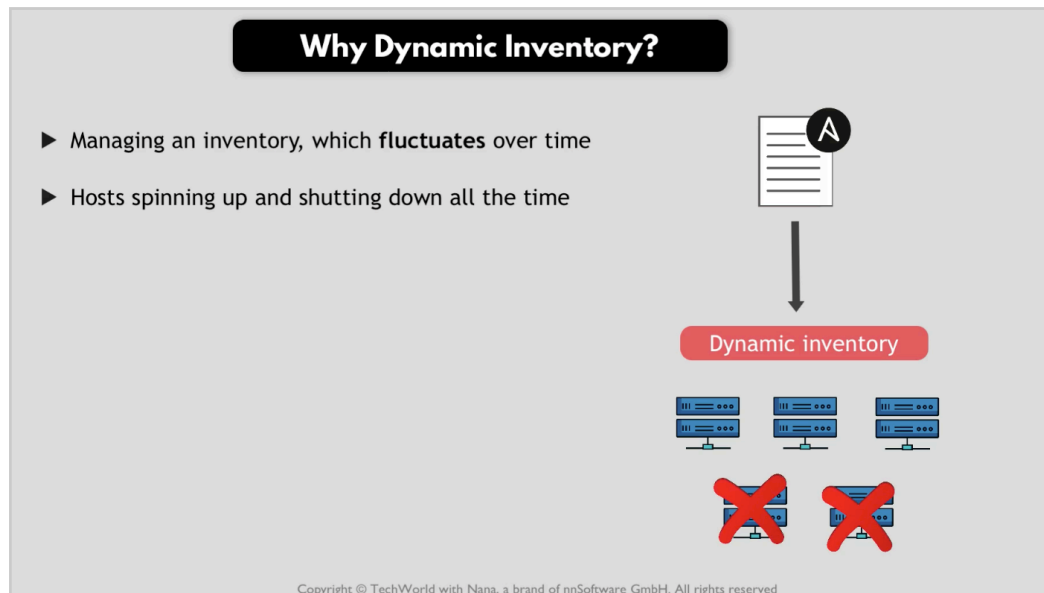
Module 15 - chapter 19

Dynamic Inventory for EC2 Servers

- The Terraform project used in this lecture can be found here: <https://gitlab.com/twn-devops-bootcamp/latest/15-ansible/terraform-learn/-/tree/feature/deploy-to-ec2>
- The final YAML file used in this lecture can be found here: <https://gitlab.com/twn-devops-bootcamp/latest/15-ansible/ansible-projects/-/blob/main/deploy-to-k8s.yaml>

Why dynamic inventory?

Let's say we have an infra that we are managing with Ansible and the infra is very dynamic -> new servers get added and deleted all the time



This is a common scenario, for example when auto-scaling depending on demand.

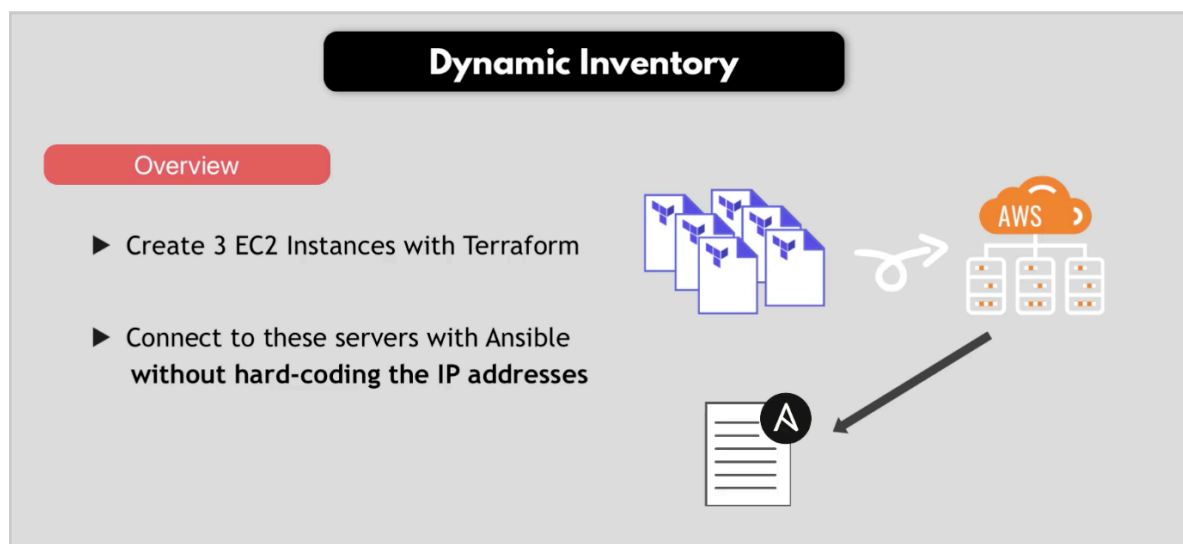
So having hardcoded IP address is not efficient

```
! deploy-docker-new-user.yaml U  hosts M X
1 [nexus_server]
2 64.226.94.75 ansible_ssh_private_key_file=~/.ssh/id_rsa ansible_user=root
3
4 [docker_server]
5 3.75.175.76 ansible_ssh_private_key_file=~/.ssh/id_rsa ansible_user=ec2-user
```

Hard-coding the IP addresses is not a good idea!

We want to dynamically set them

In this demo we will create 3 EC2 instances with TF and we will connect them to Ansible without hardcoding the IP address



Terraform - Create EC2 Instances

We add two more EC2 instances to be created by terraform

```

+resource "aws_instance" "myapp-server-one" {...}

+resource "aws_instance" "myapp-server-two" {...}

+resource "aws_instance" "myapp-server-three" {...}

```

-> terraform apply

Apply complete! Resources: 9 added, 0 changed, 0 destroyed.

Outputs:

```

aws-ami_id = "ami-01c952cfc86b7870d"
ec2-public_ip_1 = "18.134.245.218"
ec2-public_ip_2 = "18.170.39.85"
ec2-public_ip_3 = "18.130.100.205"

```

Instances (3) info									
Saved filter sets Choose filter set ▼		Find Instance by attribute or tag (case-sensitive)							
	Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS	Public IPv4 ...	Elastic IP
☐	dev-server-1	i-0972b32a6d30d9182	Running	t3.small	⌚ Initializing	eu-west-2a	ec2-18-134-245-218.eu...	18.134.245.218	-
☐	dev-server-3	i-0ec768d87acb6b6ef	Running	t3.small	⌚ Initializing	eu-west-2a	ec2-18-130-100-205.eu...	18.130.100.205	-
☐	dev-server-2	i-0058d01e46cfa3dab	Running	t3.small	⌚ Initializing	eu-west-2a	ec2-18-170-39-85.eu-w...	18.170.39.85	-

And now we move to Ansible, here we want to get the IP address directly from AWS dynamically.

As the servers can vary we don't know how many servers there are in the AWS account, we don't know their IP address. All we know is that there are some servers and we want to configure them with our Ansible playbooks.

Inventory Plugin vs Inventory Script

We need ansible to fetch the information of the EC2 instances from an specific region

Functionality that

- ▶ Connects to AWS account
- ▶ Gets server information



so we need functionality in ansible to connect to AWS and get the server info and for that we have two options:

- Dynamic Inventory plugins
- Dynamic inventory scripts

Working with dynamic inventory

- 1) Inventory Plugins
- 2) Inventory Scripts

https://docs.ansible.com/projects/ansible/latest/inventory_guide/intro_dynamic_inventory.html

A screenshot of a web browser displaying the Ansible documentation page titled "Working with dynamic inventory". The page is part of the "Building Ansible inventories" section. It includes a sidebar with navigation links like "Getting started with Execution Environments", "INSTALLATION, UPGRADE & CONFIGURATION", and "USING ANSIBLE". The main content area discusses the importance of dynamic inventory for tracking hosts from multiple sources and recommends using inventory plugins over scripts. It also mentions the "Inventory script example: Cobbler" and provides instructions on how to integrate it with Ansible. The page is marked as the "latest" version.

The documentation says that Ansible recommends inventory plugins over

inventory scripts for dynamic inventory. Because plugin can use the built-in ansible features like 'statement management' which is difficult to do with scripts.

Working with dynamic inventory



Plugins over scripts



- ▶ Inventory plugins make use of Ansible features
- ▶ STATE MANAGEMENT!

The inventory plugins are written in YAML
The inventory script are written in python

Working with dynamic inventory



Plugins over scripts



- ▶ Inventory plugins make use of Ansible features
- ▶ Inventory plugins written in **YAML**
- ▶ Inventory scripts written in **Python**



You can still use inventory scripts if you choose. When we implemented inventory plugins, we ensured backwards

So we will go with the inventory plugins as recommended.

If we want to use an inventory plugin (or script) I need to use one that is specific for the infra provider in our demo case is AWS

Plugins/Scripts specific to infrastructure provider

- ▶ You can use "*ansible-doc -t inventory -l*" to see list of available plugins
- ▶ For AWS Infrastructure, you need a specific plugin/script for AWS



"aws_ec2" Plugin

https://docs.ansible.com/projects/ansible/latest/collections/amazon/aws/aws_ec2_inventory.html

In the documentation we can see the requirements

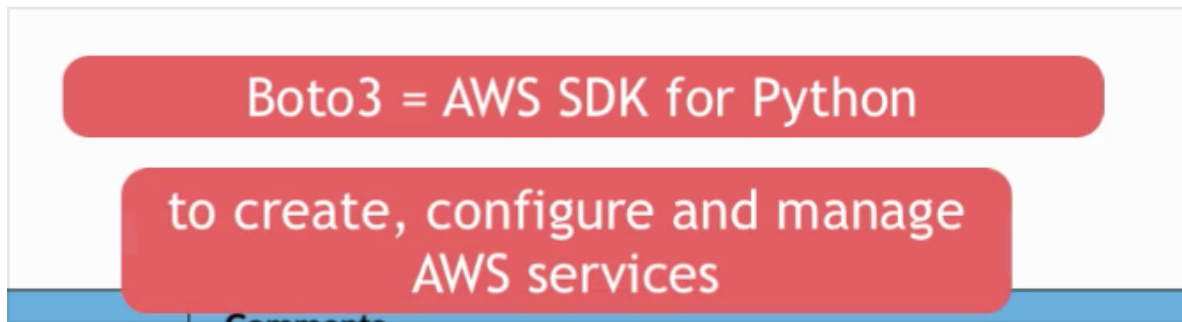
Requirements

The below requirements are needed on the local controller node that executes this **inventory**.

- python >= 3.6
- boto3 >= 1.35.0
- botocore >= 1.35.0

Parameters

Parameter	Comments
<code>access_key</code> aliases: <code>aws_access_key_id</code> , <code>aws_access_key</code> string	AWS access key ID. See the AWS documentation for more information about access keys. The <code>aws_access_key</code> and <code>profile</code> options are mutually exclusive.



Checking I have boto3

```
Terminal: Local x + v
> python3 -c "import sys; print(sys.executable)"
/usr/local/opt/python@3.14/bin/python3.14

~/Pyt/Module_15_Chapter_18_Terraform Module_15/An...19_Terraform +1 !2 ?2
```

I installed it in Module 14 - chapter 2 but it got installed in that project's virtual env -> venv. So only that project can access boto3.

-> /usr/local/opt/python@3.14/bin/python3.14 — that's the Homebrew global Python, not a venv path. If this terminal is not showing a (venv) prefix in the prompt, PyCharm isn't auto-activating the virtualenv in its terminal (a common settings default).

I opened the python project from Module 14 - chapter 2 and execute in the terminal

-> `python3 -c "import sys; print(sys.executable)"`

And I got:

`/Users/astrid/PythonProject/Automation_with_python/.venv/bin/python3`

it's correctly using the project venv where boto3 lives (.venv/bin/python3). So imports work fine here and in your scripts.

In that project the venv is activated -> activating the venv doesn't install anything, it just points that terminal session to a Python that already has boto3 installed on disk (in .venv/lib/python3.14/site-packages/). boto3 is already installed there permanently, from your pip install boto3 earlier.

So:

- Closing the terminal/session: no effect, boto3 stays installed in .venv.
- Next time you open a new terminal for this project and activate .venv again (or just use PyCharm's built-in terminal, which auto-activates it), import boto3 works immediately — no reinstall needed.

- The only thing that goes away when you close the session is the activation itself (the (.venv) prefix / PATH pointing at that Python) — you'd just re-run `source .venv/bin/activate` next time you want that terminal to use it.

Think of it as: installation is permanent (on disk), activation is per-session (just a PATH switch).

My issue:

Basically, I installed boto3 in a project in the past. so if I want to use boto3 in another project I need to install boto3 in the new project unless I install boto3 from `item` which will then be a global installation.

A:

Right, that's the core idea. Each project's .venv is isolated — packages installed in `Automation_with_python/.venv` only exist there. Two options for a new project:

1. Install boto3 into that project's own venv (recommended):

```
cd /path/to/new-project
```

```
source .venv/bin/activate
```

```
pip install boto3
```

1. Keeps dependencies isolated per project — the standard practice, since different projects can need different (even conflicting) package versions.

2. Install boto3 globally (e.g. via `pip3 install boto3` in iTerm without activating any venv) — it goes into the system Python (`/usr/local/opt/python@3.14/.../site-packages`), and then any project that uses that same global Python (i.e., doesn't have its own venv, or its venv wasn't created with `--system-site-packages`) would see it too.

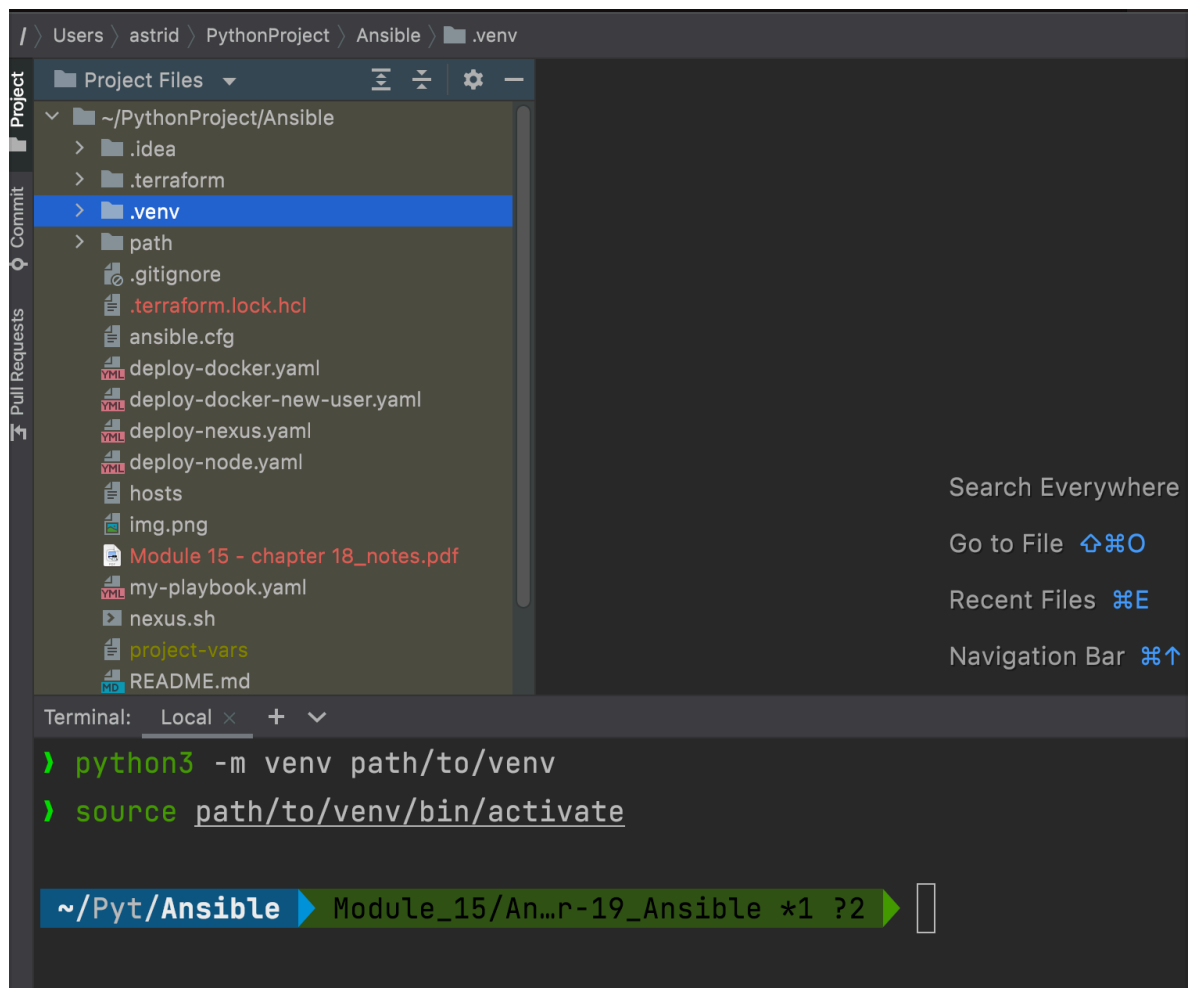
The tradeoff: global installs are convenient but get messy fast — you lose track of what each project actually needs, and version conflicts between projects become a real headache. Per-project venvs are the standard for a reason. Given you're already doing venvs per project (as with `Automation_with_python`), I'd stick with option 1 for consistency.

So I will install boto3 in the new project.

```
-> python3 -m venv path/to/venv
```

```
-> source .venv/bin/activate
```

Now my project has the venv activated (it was not activated as I deleted that folder before, I thought I didn't need it)



-> pip install boto3

```

> pip install boto3
Collecting boto3
  Downloading boto3-1.43.68-py3-none-any.whl.metadata (6.6 kB)
Collecting botocore<1.44.0,>=1.43.68 (from boto3)
  Downloading botocore-1.43.68-py3-none-any.whl.metadata (5.6 kB)
Collecting jmespath<2.0.0,>=0.7.1 (from boto3)
  Using cached jmespath-1.1.0-py3-none-any.whl.metadata (7.6 kB)
Collecting s3transfer<0.20.0,>=0.19.0 (from boto3)
  Downloading s3transfer-0.19.2-py3-none-any.whl.metadata (1.7 kB)
Collecting python-dateutil<3.0.0,>=2.1 (from botocore<1.44.0,>=1.43.68->boto3)
  Using cached python_dateutil-2.9.0.post0-py2.py3-none-any.whl.metadata (8.4 kB)
Collecting urllib3!=2.2.0,<3,>=1.25.4 (from botocore<1.44.0,>=1.43.68->boto3)
  Using cached urllib3-2.7.0-py3-none-any.whl.metadata (6.9 kB)
Collecting six>=1.5 (from python-dateutil<3.0.0,>=2.1->botocore<1.44.0,>=1.43.68->boto3)
  Using cached six-1.17.0-py2.py3-none-any.whl.metadata (1.7 kB)
Downloading boto3-1.43.68-py3-none-any.whl (140 kB)
Downloading botocore-1.43.68-py3-none-any.whl (15.6 MB)
  ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 15.6/15.6 MB 28.6 MB/s 0:00:00
Using cached jmespath-1.1.0-py3-none-any.whl (20 kB)
Using cached python_dateutil-2.9.0.post0-py2.py3-none-any.whl (229 kB)
Downloading s3transfer-0.19.2-py3-none-any.whl (90 kB)
Using cached urllib3-2.7.0-py3-none-any.whl (131 kB)
Using cached six-1.17.0-py2.py3-none-any.whl (11 kB)

```

```

Installing collected packages: urllib3, six, jmespath, python-dateutil, botocore, s3transfer, boto3
Successfully installed boto3-1.43.68 botocore-1.43.68 jmespath-1.1.0 python-dateutil-2.9.0.post0 s3transfer-0.19.2 six-1.17.0 u
rllib3-2.7.0

```

Q:

and now how do i check that boto3 is installed in the venv

A:

With the venv activated, any of these:

`pip show boto3`

or

`pip list | grep boto3`

or the direct import check:

`python3 -c "import boto3; print(boto3.__version__)"`

If the venv is active you'll see (.venv) in your prompt — make sure that's showing, otherwise you're checking the wrong Python again.

```

> pip show boto3
Name: boto3
Version: 1.43.68
Summary: The AWS SDK for Python
Home-page: https://github.com/boto/boto3
Author: Amazon Web Services
Author-email:
License: Apache-2.0
Location: /Users/astrid/PythonProject/Ansible/.venv/lib/python3.14/site-packages
Requires: botocore, jmespath, s3transfer
Required-by:
> python3 -c "import boto3; print(boto3.__version__)"
1.43.68

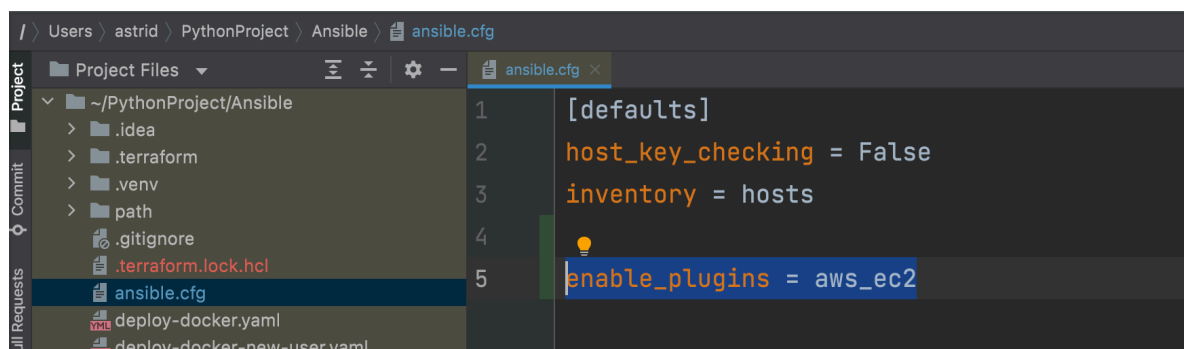
```

So now my ansible project has boto3 and botocore installed 

Now let's tell ansible the name of the plugin



For that we enable the plugin in the `ansible.cfg` file with 'enable_plugins' (it could be a list of plugins)

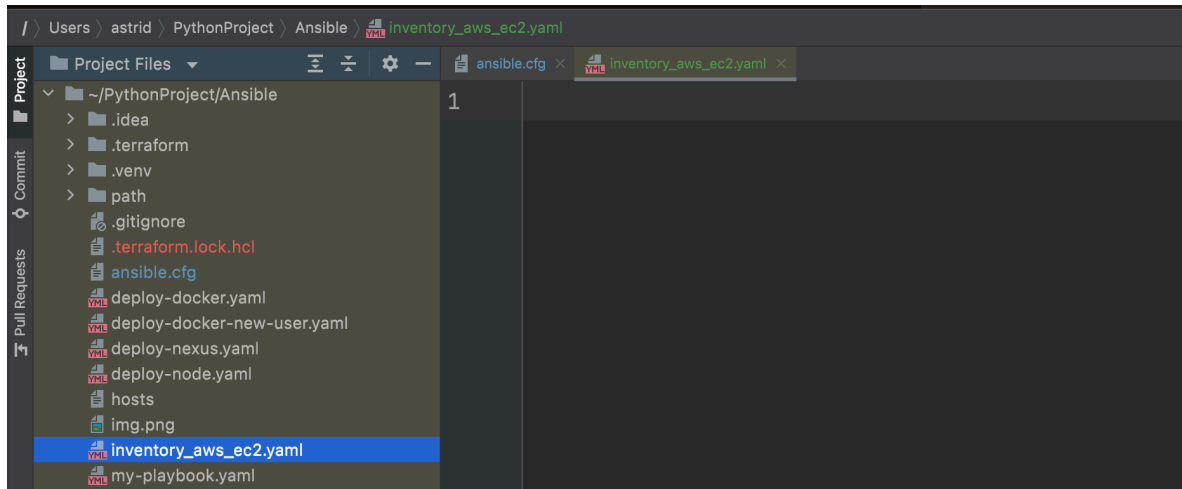


Write plugin configuration

Create a file for the plugin configuration and we will call it 'inventory_aws_ec2.yaml' and 'aws_ec2' part is mandatory in the name for the plugin to be recognised.

EC2 inventory source

- Config file must end with "aws_ec2.yaml"



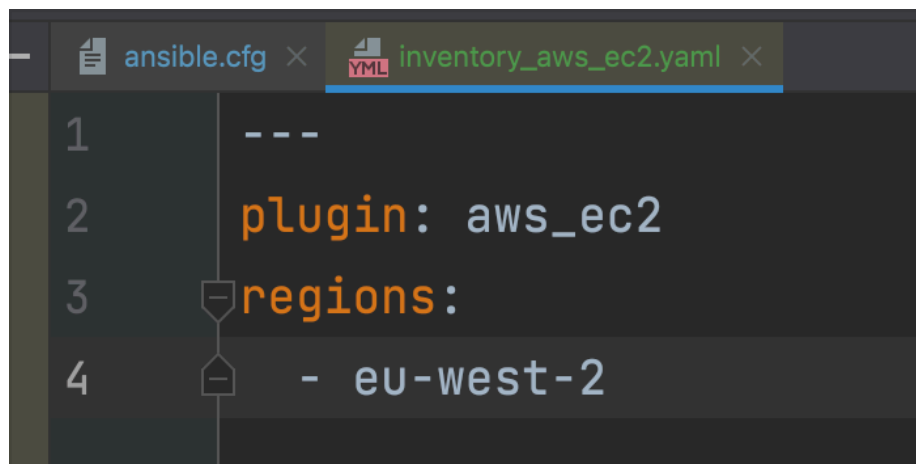
Now let's write the plugin configuration using the yaml format, and we start with the plugin's name, followed by the attributes for the plugin:

- regions -> AWS region(s) we want to check our inventory from.

EC2 inventory source

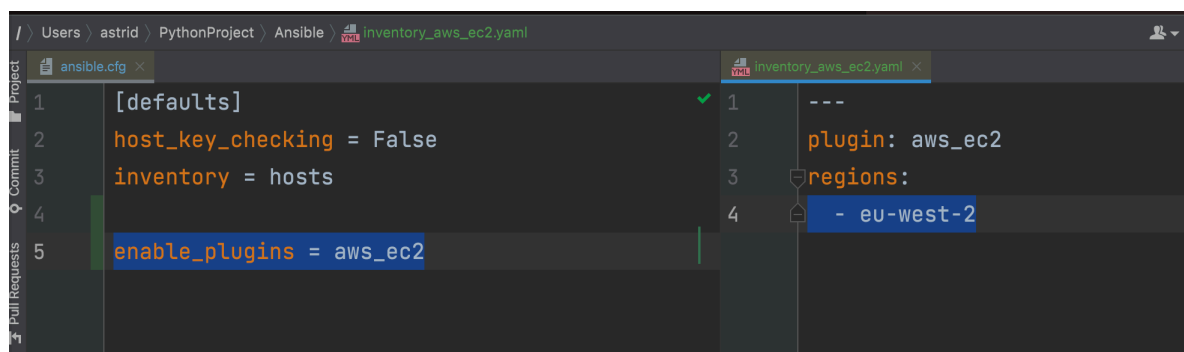
- Get all hosts from specific region(s)

We only have one region where our instances are running 'eu-west-2'



```
1 ---
2 plugin: aws_ec2
3 regions:
4   - eu-west-2
```

So we have enabled the plugin in ansible.cfg and then we configured that plugin to tell it which region(s) to connect to.



```
1 [defaults]
2 host_key_checking = False
3 inventory = hosts
4
5 enable_plugins = aws_ec2
```

```
1 ---
2 plugin: aws_ec2
3 regions:
4   - eu-west-2
```

With this config we can execute our plugin separately without any playbook execution and see the inventory list that it returns:

-> ansible-inventory



ansible-inventory

► Used to display or dump the configured inventory as Ansible sees it

And we pass an argument which is the plugins configuration yaml file

-> ansible-inventory -i inventory_aws_ec2.yaml --list

We get back the information of the three EC2 instances

```

    "profile": "inventory_legacy"
  },
  "all": {
    "children": [
      "ungrouped",
      "aws_ec2"
    ]
  },
  "aws_ec2": {
    "hosts": [
      "ec2-18-170-39-85.eu-west-2.compute.amazonaws.com",
      "ec2-18-134-245-218.eu-west-2.compute.amazonaws.com",
      "ec2-18-130-100-205.eu-west-2.compute.amazonaws.com"
    ]
  }
}

```

At this point we are not connected to the servers yet, we haven't ssh into them for that we need the private key and server user and we have not provided that info yet.

We have only connected to the AWS account using boto3 to fetch the info of the servers.

Q:

I have provisioned infra to aws using terraform, now I have use the ansible plugin `aws_ec2` to connect to the aws account and fetch the servers info, why did that work? how boto3 knew how to connect to my AWS account?

A:

I ran in previous demos
-> aws configure

which wrote my access key/secret into `~/.aws/credentials`. Terraform's AWS provider uses this exact same chain to authenticate — so does the AWS CLI. boto3 just found the same credentials sitting there and used them, with no extra setup needed on my end.

Ansible's `aws_ec2` inventory plugin is literally a thin wrapper around boto3/botocore under the hood, so it inherits this same auto-discovery — that's why it "just worked" without you passing keys into your playbook or inventory file.

Because boto3/botocore (and anything built on it, including Ansible's `aws_ec2` inventory plugin) doesn't need credentials handed to it explicitly — it searches a standard, ordered chain of places for them automatically:

1. Explicit params in code (e.g. `boto3.client('ec2', aws_access_key_id=..., aws_secret_access_key=...)`) — not used in your case.

2. Environment variables — AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY, AWS_SESSION_TOKEN.
3. Shared credentials file — ~/.aws/credentials (and ~/.aws/config for region/profile settings).
4. SSO / assume-role config in ~/.aws/config, if you use profiles.
5. ECS/container credentials — if running inside an ECS task.
6. EC2 instance metadata service — if running on an EC2 instance with an attached IAM role.

Currently the output return has too much info and we only need the “hosts” information, so we can use ‘graph’ instead of ‘list’

-> ansible-inventory -i inventory_aws_ec2.yaml --graph

```
> ansible-inventory -i inventory_aws_ec2.yaml --graph
[WARNING]: Deprecation warnings can be disabled by setting `deprecation_warnings=False` in ansible.cfg.
[DEPRECATION WARNING]: The 'tags' host variable is deprecated. Use 'ec2_tags' instead. This feature will be removed from collection 'amazon.aws' in a release after 2026-12-01.
@all:
  |--@ungrouped:
  |--@aws_ec2:
    |--ec2-18-170-39-85.eu-west-2.compute.amazonaws.com
    |--ec2-18-134-245-218.eu-west-2.compute.amazonaws.com
    |--ec2-18-130-100-205.eu-west-2.compute.amazonaws.com
```

— tangent on the two messages returned by ansible 🙌

Two separate messages, both harmless right now but worth understanding:

First line — just informational. Ansible prints deprecation warnings by default; it's telling you how to silence them (deprecation_warnings=False in ansible.cfg) if they get noisy. Not something to act on.

Second line — the actual deprecation — the amazon.aws collection (which provides the aws_ec2 inventory plugin) currently exposes each EC2 instance's tags as a host variable literally called tags. That's being renamed to ec2_tags because tags is a generic name that risks colliding with other variables/plugins. Support for the old tags name will be removed in a release after 2026-12-01 — until then it still works, just warns.

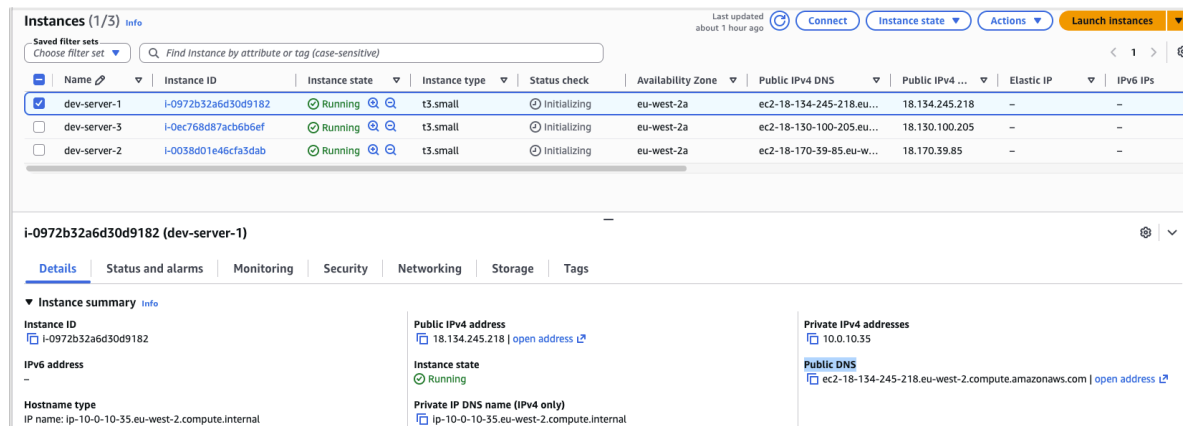
It only matters to you if somewhere in your playbooks/templates you reference {{ tags.Name }} or similar (pulled from this inventory). Check with:

```
grep -rn "tags\" /Users/astirid/PythonProject/Ansible/*.yaml
```

If you find usages sourced from the EC2 inventory, switch them to ec2_tags.Name etc. now so you're not caught out when the old name is removed. If you're not referencing tags anywhere yet, no action needed — just informational for now.

— end of tangent —

We can see that it returns the DNS name of each EC2 instance

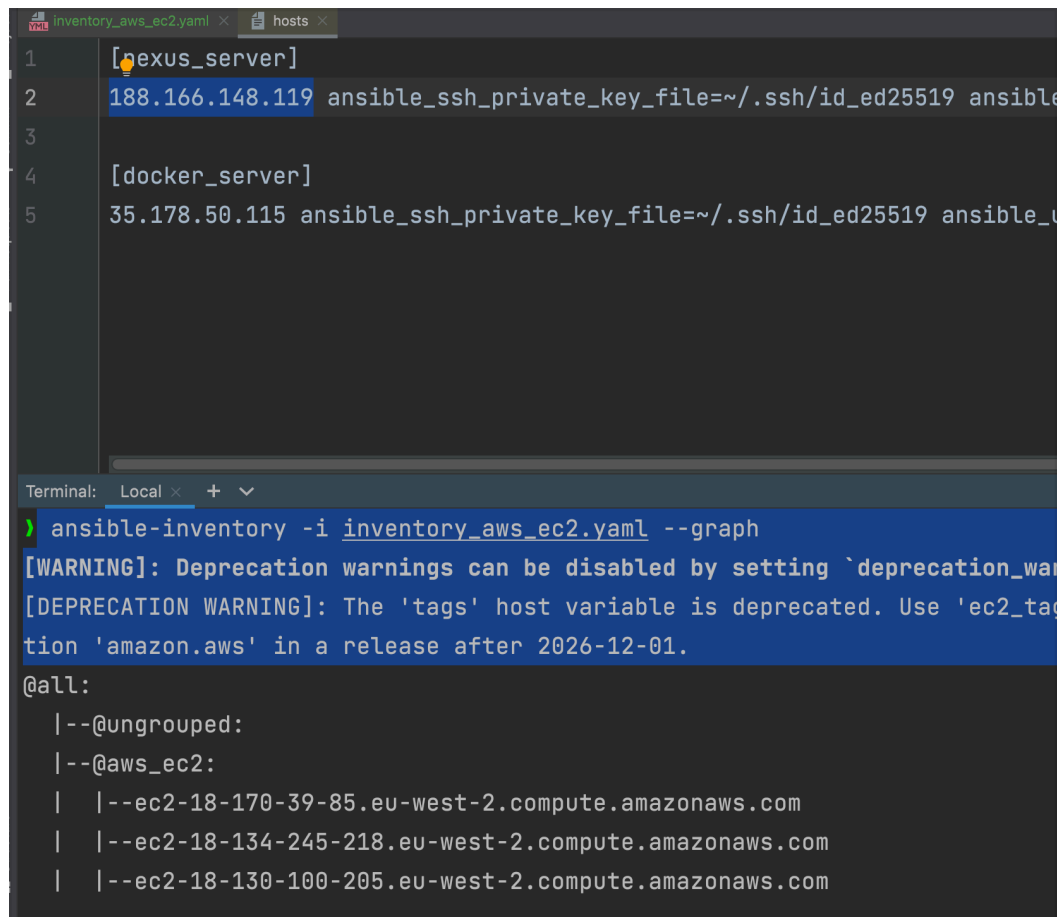


The screenshot shows the AWS Management Console 'Instances' page. It lists three EC2 instances: dev-server-1, dev-server-3, and dev-server-2. Below the list, the details for instance i-0972b32a6d30d9182 (dev-server-1) are expanded, showing its status as 'Running' and its public IP address as 18.134.245.218.

Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS	Public IPv4 ...	Elastic IP	IPv6 IPs
dev-server-1	i-0972b32a6d30d9182	Running	t3.small	Initializing	eu-west-2a	ec2-18-134-245-218.eu...	18.134.245.218	-	-
dev-server-3	i-0ec768d87acb6b6ef	Running	t3.small	Initializing	eu-west-2a	ec2-18-130-100-205.eu...	18.130.100.205	-	-
dev-server-2	i-0038d01e46cfa3dab	Running	t3.small	Initializing	eu-west-2a	ec2-18-170-39-85.eu-w...	18.170.39.85	-	-

i-0972b32a6d30d9182 (dev-server-1)		
Instance summary	Public IPv4 address 18.134.245.218 open address	Private IPv4 addresses 10.0.10.35
Instance ID i-0972b32a6d30d9182	Instance state Running	Public DNS ec2-18-134-245-218.eu-west-2.compute.amazonaws.com open address
IPv6 address -	Private IP DNS name (IPv4 only) ip-10-0-10-35.eu-west-2.compute.internal	
Hostname type IP name: ip-10-0-10-35.eu-west-2.compute.internal		

So it is the equivalent of having the 'hosts' file but this time is dynamic.



The screenshot shows a terminal window with an Ansible inventory file and the output of an Ansible command. The inventory file 'inventory_aws_ec2.yaml' defines two hosts: 'nexus_server' with IP 188.166.148.119 and 'docker_server' with IP 35.178.50.115. The terminal output shows the command 'ansible-inventory -i inventory_aws_ec2.yaml --graph' and its output, which lists the hosts and their associated IP addresses.

```
1 [nexus_server]
2 188.166.148.119 ansible_ssh_private_key_file=~/.ssh/id_ed25519 ansible_
3
4 [docker_server]
5 35.178.50.115 ansible_ssh_private_key_file=~/.ssh/id_ed25519 ansible_u

Terminal: Local x + v
> ansible-inventory -i inventory_aws_ec2.yaml --graph
[WARNING]: Deprecation warnings can be disabled by setting `deprecation_war
[DEPRECATION WARNING]: The 'tags' host variable is deprecated. Use 'ec2_tag
tion 'amazon.aws' in a release after 2026-12-01.
@all:
  |--@ungrouped:
  |--@aws_ec2:
    |--ec2-18-170-39-85.eu-west-2.compute.amazonaws.com
    |--ec2-18-134-245-218.eu-west-2.compute.amazonaws.com
    |--ec2-18-130-100-205.eu-west-2.compute.amazonaws.com
```

(The IP dresses/DNS of 'hosts' file don't match what is returned by the plugin because they are old configuration for other infra where we were hardcoding the IP addresses)

Now with the DNS of the instances fetched, the playbook will be able to connect to the instances.

Assign public DNS to EC2 instances

Nana got the private DNS not the public (which is my case)

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

```
nana@macbook /Users/nana/ansible [master]
• % ansible-inventory -i inventory_aws_ec2.yaml --graph
@all:
  |--@ungrouped:
  |--@aws_ec2:
  |   |--ip-10-0-10-149.eu-central-1.compute.internal
  |   |--ip-10-0-10-42.eu-central-1.compute.internal
  |   |--ip-10-0-10-241.eu-central-1.compute.internal
nana@macbook /Users/nana/ansible [master]
```

PRIVATE

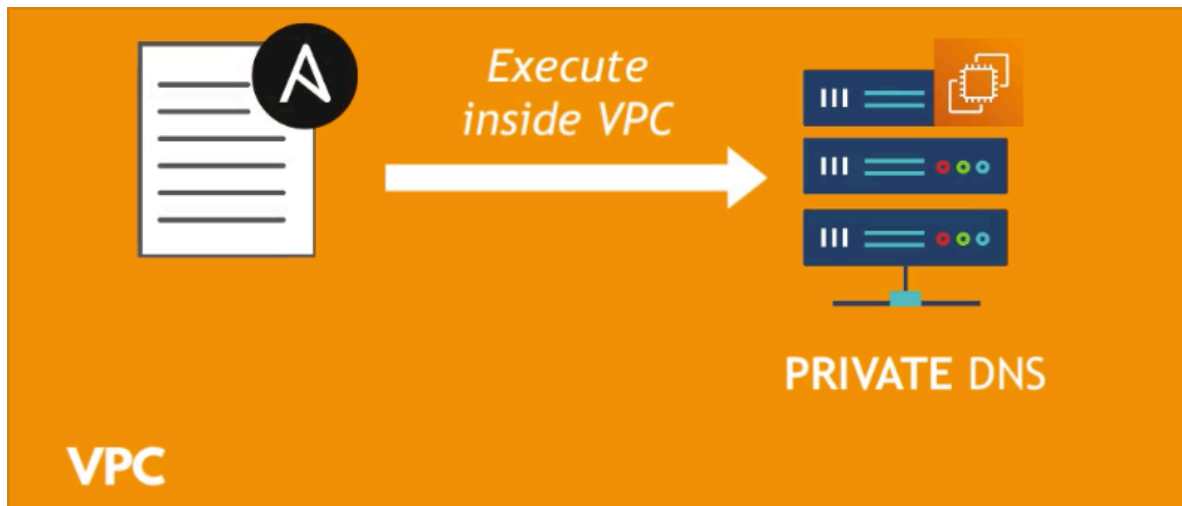
And she got the private because her instances didn't get public DNS assign

Instances (1/3) Info							
Find Instance by attribute or tag (case-sensitive)				Any state			
Instance state = running				Clear filters			
☐	Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone
☐	dev-server-three	i-09d3dd637a9c42285	Running	t2.small	Initializing	View alarms +	eu-central-1a
☐	dev-server-two	i-0c97a079d32c015a4	Running	t2.micro	Initializing	View alarms +	eu-central-1a
☒	dev-server	i-0703076dd31d07e05	Running	t2.micro	Initializing	View alarms +	eu-central-1a

Instance: i-0703076dd31d07e05 (dev-server)

Instance ID i-0703076dd31d07e05 (dev-server)	Public IPv4 address 18.159.130.186 open address	Private IPv4 addresses 10.0.10.149
IPv6 address -	Instance state Running	Public IPv4 DNS
Hostname type IP name: ip-10-0-10-149.eu-central-1.compute.internal	Private IP DNS name (IPv4 only) ip-10-0-10-149.eu-central-1.compute.internal	
Answer private resource DNS name -	Instance type t2.micro	Elastic IP addresses -

If I use the private DNS to connect to the servers then I need to execute the ansible command from inside the VPC where the servers are running



If we want to connect to the servers of a VPC from outside the VPC like our laptop we need the public IP address/DNS



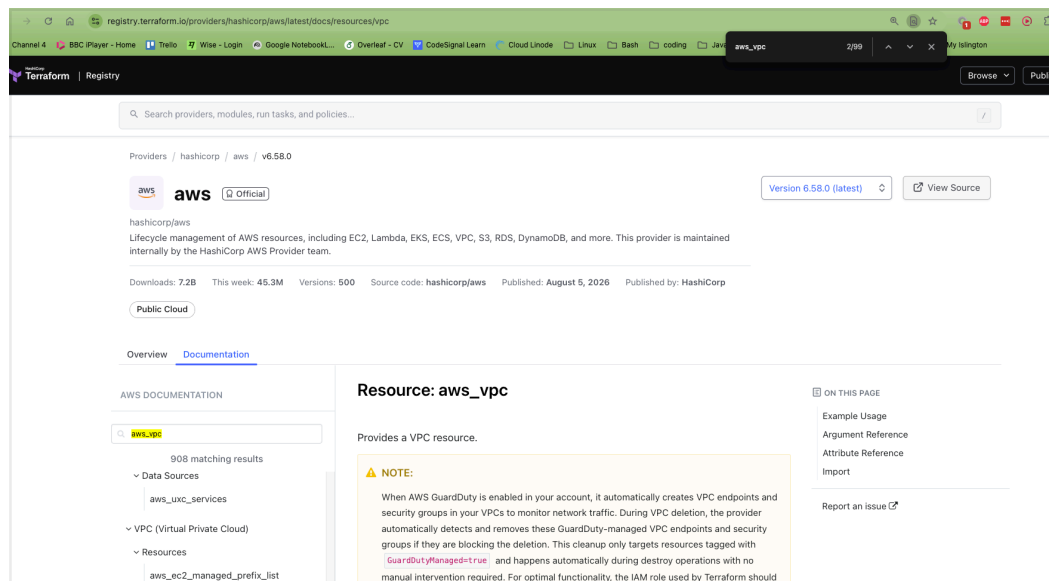
So terraform configuration needs an edition in order to provide a public IP DNS when provisioning the servers.

I need to use 'enable_dns_hostnames = true' see line 17 📌

```
15 resource "aws_vpc" "myapp-vpc" {
16     cidr_block = var.vpc_cidr_block
17     enable_dns_hostnames = true
18     tags = {
19         Name: "${var.env_prefix}-vpc"
20     }
21 }
```

And I already have that code, that is why it works for me already.

Here is the documentation



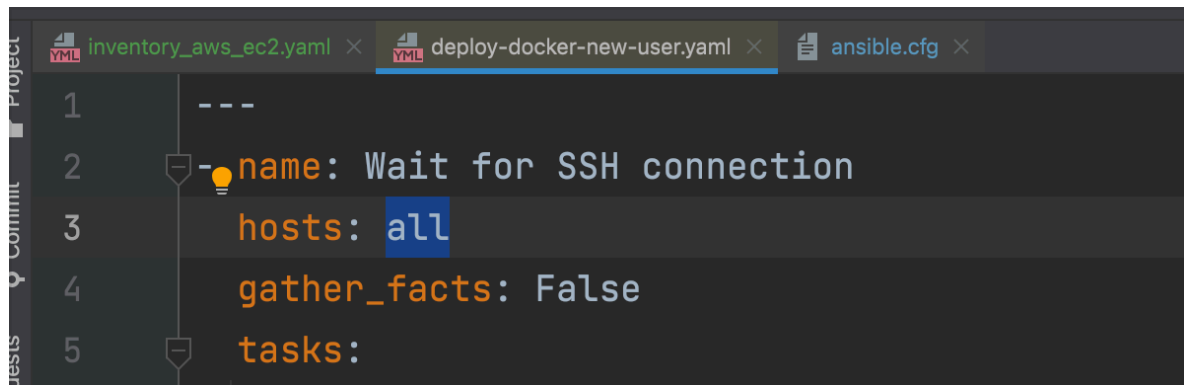
aws_ec2_traffic_mirror_filter	region of the VPC. This can be changed to restrict advertisement of public addresses to specific Network Border Groups such as LocalZones.	Report an
aws_ec2_traffic_mirror_filter_rule		
aws_ec2_traffic_mirror_session		
aws_ec2_traffic_mirror_target		
aws_security_group		
aws_security_group_rule		
• aws_vpc		
aws_vpc_block_public_access_exclusion		
aws_vpc_block_public_access_options		

Next, let's configure ansible to use the fetched inventory in the playbook.

Configure Ansible to use dynamic inventory

So instead of the 'hosts' file we want ansible to use the inventory plugin and we need to edit the playbook field 'hosts'

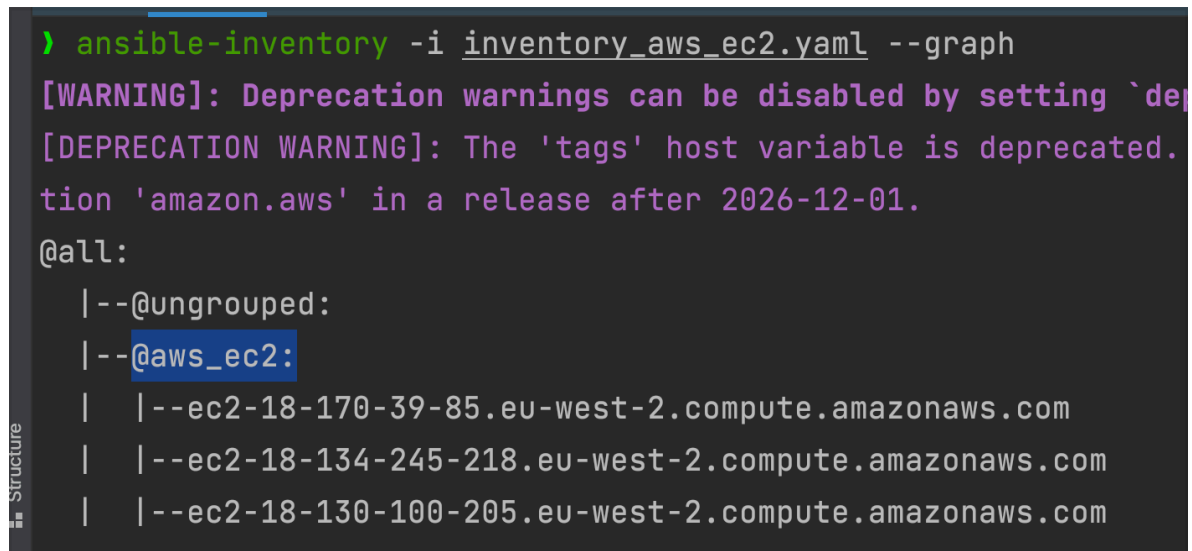
We can keep using 'all'



The screenshot shows an IDE with three tabs: `inventory_aws_ec2.yaml`, `deploy-docker-new-user.yaml`, and `ansible.cfg`. The `deploy-docker-new-user.yaml` tab is active, displaying the following Ansible task:

```
1 ---
2 - name: Wait for SSH connection
3   hosts: all
4   gather_facts: False
5   tasks:
```

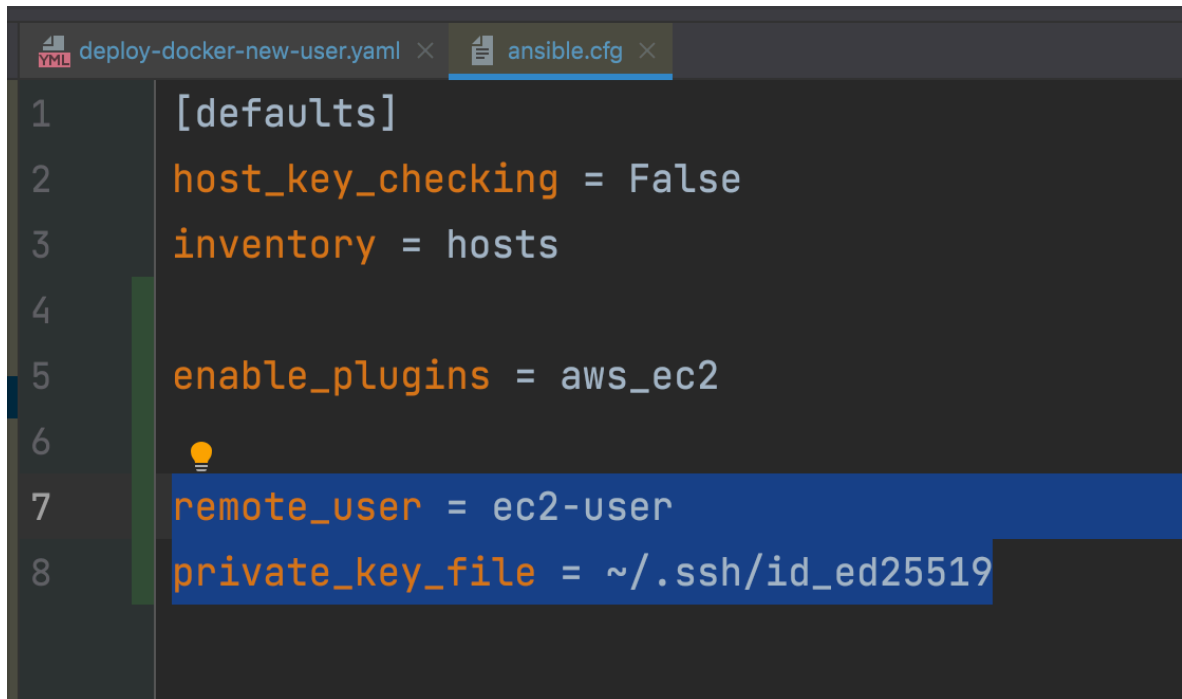
Or we can use the name of the group fetched by ansible inventory plugin



The screenshot shows a terminal window with the following output:

```
> ansible-inventory -i inventory_aws_ec2.yaml --graph
[WARNING]: Deprecation warnings can be disabled by setting `deprecation_warnings=False`
[DEPRECATION WARNING]: The 'tags' host variable is deprecated.
                        This message will be removed after a release after 2026-12-01.
@all:
  |--@ungrouped:
  |--@aws_ec2:
    |--ec2-18-170-39-85.eu-west-2.compute.amazonaws.com
    |--ec2-18-134-245-218.eu-west-2.compute.amazonaws.com
    |--ec2-18-130-100-205.eu-west-2.compute.amazonaws.com
```

So I update the 'hosts'



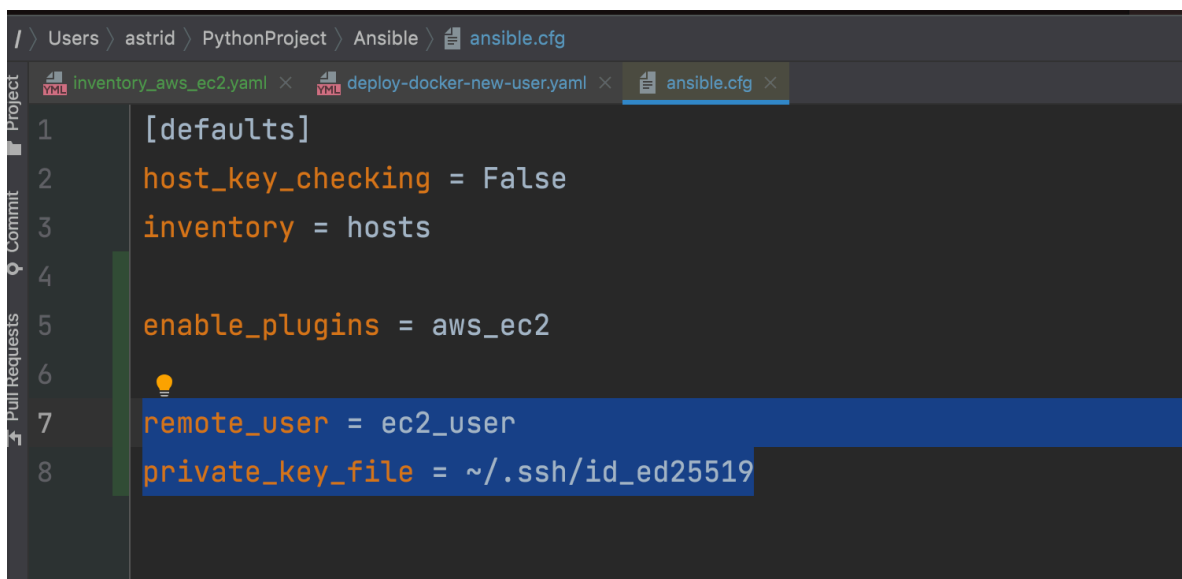
```
1 [defaults]
2   host_key_checking = False
3   inventory = hosts
4
5   enable_plugins = aws_ec2
6
7   remote_user = ec2-user
8   private_key_file = ~/.ssh/id_ed25519
```

And whenever we are connecting to the host we need:

- User
- Private ssh key

We can set them globally in the ansible.cfg, we also can pass them as argument in the terminal when executing -> ansible-playbook.

We we set them up in the ansible.cfg file in this demo



```
1 [defaults]
2   host_key_checking = False
3   inventory = hosts
4
5   enable_plugins = aws_ec2
6
7   remote_user = ec2_user
8   private_key_file = ~/.ssh/id_ed25519
```

Now to execute the playbook we pass the playbook we want to execute and also we pass the flag '-i' to pass the dynamic inventory along with the inventory configuration file

-> ansible-playbook -i inventory_aws_ec2.yaml deploy-docker-new-user.yaml

```
Terminal: Local x + v
> ansible-playbook -i inventory_aws_ec2.yaml deploy-docker-new-user.yaml
[WARNING]: Deprecation warnings can be disabled by setting `deprecation_warnings=False` in ansible.cfg.
[DEPRECATION WARNING]: The 'tags' host variable is deprecated. Use 'ec2_tags' instead. This feature will be
tion 'amazon.aws' in a release after 2026-12-01.

PLAY [Wait for SSH connection] *****
[WARNING]: Found variable using reserved name 'tags'.
Origin: <unknown>

tags

TASK [Ensure SSH port open] *****
ok: [ec2-18-170-39-85.eu-west-2.compute.amazonaws.com]
ok: [ec2-18-134-245-218.eu-west-2.compute.amazonaws.com]
ok: [ec2-18-130-100-205.eu-west-2.compute.amazonaws.com]
```

```
PLAY [Install Docker] *****

TASK [Gathering Facts] *****
[WARNING]: Host 'ec2-18-134-245-218.eu-west-2.compute.amazonaws.com' is using the discovered Python interpreter at '/usr/bin/py
thon3.9', but future installation of another Python interpreter could cause a different interpreter to be discovered. See https://docs.ansible.com/ansible-core/2.21/reference\_appendices/interpreter\_discovery.html for more information.
ok: [ec2-18-134-245-218.eu-west-2.compute.amazonaws.com]
[WARNING]: Host 'ec2-18-170-39-85.eu-west-2.compute.amazonaws.com' is using the discovered Python interpreter at '/usr/bin/py
thon3.9', but future installation of another Python interpreter could cause a different interpreter to be discovered. See https://docs.ansible.com/ansible-core/2.21/reference\_appendices/interpreter\_discovery.html for more information.
ok: [ec2-18-170-39-85.eu-west-2.compute.amazonaws.com]
[WARNING]: Host 'ec2-18-130-100-205.eu-west-2.compute.amazonaws.com' is using the discovered Python interpreter at '/usr/bin/py
thon3.9', but future installation of another Python interpreter could cause a different interpreter to be discovered. See https://docs.ansible.com/ansible-core/2.21/reference\_appendices/interpreter\_discovery.html for more information.
ok: [ec2-18-130-100-205.eu-west-2.compute.amazonaws.com]

TASK [Make sure Docker is installed] *****
```

```
TASK [Make sure Docker is installed] *****
changed: [ec2-18-134-245-218.eu-west-2.compute.amazonaws.com]
changed: [ec2-18-170-39-85.eu-west-2.compute.amazonaws.com]
changed: [ec2-18-130-100-205.eu-west-2.compute.amazonaws.com]

TASK [Ensure Docker is running] *****
changed: [ec2-18-170-39-85.eu-west-2.compute.amazonaws.com]
changed: [ec2-18-130-100-205.eu-west-2.compute.amazonaws.com]
changed: [ec2-18-134-245-218.eu-west-2.compute.amazonaws.com]

PLAY [Create new linux user] *****

TASK [Gathering Facts] *****
ok: [ec2-18-134-245-218.eu-west-2.compute.amazonaws.com]
ok: [ec2-18-170-39-85.eu-west-2.compute.amazonaws.com]
ok: [ec2-18-130-100-205.eu-west-2.compute.amazonaws.com]
```

```

TASK [Create new linux user] *****
changed: [ec2-18-134-245-218.eu-west-2.compute.amazonaws.com]
changed: [ec2-18-170-39-85.eu-west-2.compute.amazonaws.com]
changed: [ec2-18-130-100-205.eu-west-2.compute.amazonaws.com]

PLAY [Install Docker-compose] *****

TASK [Gathering Facts] *****
[WARNING]: Module remote_tmp /home/astrid/.ansible/tmp did not exist and was created with a mode of 0700, this may cause issues
when running as another user. To avoid this, create the remote_tmp dir with the correct permissions manually
ok: [ec2-18-134-245-218.eu-west-2.compute.amazonaws.com]
ok: [ec2-18-170-39-85.eu-west-2.compute.amazonaws.com]
ok: [ec2-18-130-100-205.eu-west-2.compute.amazonaws.com]

```

```

TASK [Create docker-compose directory] *****
changed: [ec2-18-134-245-218.eu-west-2.compute.amazonaws.com]
changed: [ec2-18-170-39-85.eu-west-2.compute.amazonaws.com]
changed: [ec2-18-130-100-205.eu-west-2.compute.amazonaws.com]

TASK [get architecture of remote machine] *****
changed: [ec2-18-134-245-218.eu-west-2.compute.amazonaws.com]
changed: [ec2-18-130-100-205.eu-west-2.compute.amazonaws.com]
changed: [ec2-18-170-39-85.eu-west-2.compute.amazonaws.com]

TASK [Install docker-compose] *****
changed: [ec2-18-134-245-218.eu-west-2.compute.amazonaws.com]
changed: [ec2-18-130-100-205.eu-west-2.compute.amazonaws.com]
changed: [ec2-18-170-39-85.eu-west-2.compute.amazonaws.com]

```

```

PLAY [Start docker containers] *****

TASK [Gathering Facts] *****
ok: [ec2-18-134-245-218.eu-west-2.compute.amazonaws.com]
ok: [ec2-18-130-100-205.eu-west-2.compute.amazonaws.com]
ok: [ec2-18-170-39-85.eu-west-2.compute.amazonaws.com]

TASK [Copy docker compose] *****
changed: [ec2-18-134-245-218.eu-west-2.compute.amazonaws.com]
changed: [ec2-18-170-39-85.eu-west-2.compute.amazonaws.com]
changed: [ec2-18-130-100-205.eu-west-2.compute.amazonaws.com]

TASK [Docker login] *****
changed: [ec2-18-134-245-218.eu-west-2.compute.amazonaws.com]
changed: [ec2-18-130-100-205.eu-west-2.compute.amazonaws.com]
changed: [ec2-18-170-39-85.eu-west-2.compute.amazonaws.com]

```

```

TASK [Start containers from compose] *****
changed: [ec2-18-130-100-205.eu-west-2.compute.amazonaws.com]
changed: [ec2-18-170-39-85.eu-west-2.compute.amazonaws.com]
changed: [ec2-18-134-245-218.eu-west-2.compute.amazonaws.com]

PLAY RECAP *****
ec2-18-130-100-205.eu-west-2.compute.amazonaws.com : ok=14  changed=9  unreachable=0  failed=0  skipped=0  rescued=0
ignored=0
ec2-18-134-245-218.eu-west-2.compute.amazonaws.com : ok=14  changed=9  unreachable=0  failed=0  skipped=0  rescued=0
ignored=0
ec2-18-170-39-85.eu-west-2.compute.amazonaws.com : ok=14  changed=9  unreachable=0  failed=0  skipped=0  rescued=0
ignored=0

```

So it works 🥳

Now if I add a new server to terraform

```
Module_15_Chapter_18_Terraform - main.tf
Module_15_Chapter_18_Terraform > main.tf
main.tf x
151 resource "aws_instance" "myapp-server-three" {...}
166
167 resource "aws_instance" "myapp-server-four" {
168     ami = data.aws_ami.latest-amazon-linux-image.id
169     instance_type = var.instance_type
170
171     subnet_id = aws_subnet.myapp-subnet-1.id
172     vpc_security_group_ids = [aws_default_security_group.default-sg.id]
173     availability_zone = var.avail_zone
174
175     associate_public_ip_address = true
176     key_name = aws_key_pair.ssh-key.key_name
177
178     tags = {
179         Name: "${var.env_prefix}-server-4"
180     }
181 }
```

-> terraform apply

```
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

Outputs:

aws-ami_id = "ami-01c952cfc86b7870d"
ec2-public_ip_1 = "18.134.245.218"
ec2-public_ip_2 = "18.170.39.85"
ec2-public_ip_3 = "18.130.100.205"
ec2-public_ip_4 = "13.135.103.176"
```

Instances (1/4) Info

Last updated less than a minute ago

Connect Instance state Actions Launch instances

Saved filter sets

Choose filter set

Find instance by attribute or tag (case-sensitive)

Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS	Public IPv4 ...	Elastic IP	IPv6 IPs
dev-server-1	i-0972b32a6d30d9182	Running	t3.small	3/3 checks passed	eu-west-2a	ec2-18-134-245-218.eu...	18.134.245.218	-	-
dev-server-3	i-0ec768d87ac6b66ef	Running	t3.small	3/3 checks passed	eu-west-2a	ec2-18-130-100-205.eu...	18.130.100.205	-	-
dev-server-2	i-0038d01e46cfa3dab	Running	t3.small	3/3 checks passed	eu-west-2a	ec2-18-170-39-85.eu-w...	18.170.39.85	-	-
dev-server-4	i-0d875ae88de121bbd	Running	t3.small	Initializing	eu-west-2a	ec2-13-135-103-176.eu...	13.135.103.176	-	-

i-0d875ae88de121bbd (dev-server-4)

Details Status and alarms Monitoring Security Networking Storage Tags

▼ Instance summary Info

Instance ID i-0d875ae88de121bbd	Public IPv4 address 13.135.103.176 open address	Private IPv4 addresses 10.0.10.241
IPv6 address -	Instance state Running	Public DNS ec2-13-135-103-176.eu-west-2.compute.amazonaws.com open address
Hostname type IP name: ip-10-0-10-241.eu-west-2.compute.internal	Private IP DNS name (IPv4 only) ip-10-0-10-241.eu-west-2.compute.internal	
Answer private resource DNS name	Instance type	Elastic IP addresses

And then I execute the ansible playbook again, it will find the new server and it will configure it:

-> ansible-playbook -i inventory_aws_ec2.yaml deploy-docker-new-user.yaml

```
TASK [Docker login] *****
ok: [ec2-18-134-245-218.eu-west-2.compute.amazonaws.com]
ok: [ec2-18-170-39-85.eu-west-2.compute.amazonaws.com]
ok: [ec2-18-130-100-205.eu-west-2.compute.amazonaws.com]
changed: [ec2-13-135-103-176.eu-west-2.compute.amazonaws.com]

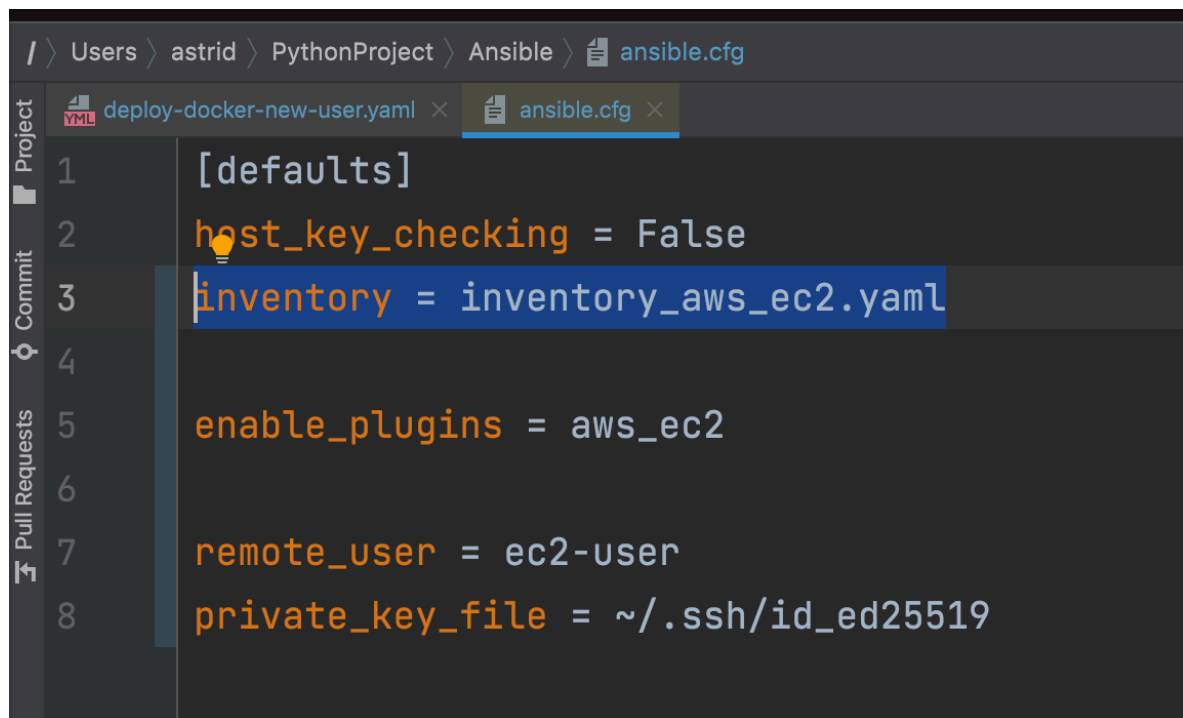
TASK [Start containers from compose] *****
ok: [ec2-18-134-245-218.eu-west-2.compute.amazonaws.com]
ok: [ec2-18-170-39-85.eu-west-2.compute.amazonaws.com]
ok: [ec2-18-130-100-205.eu-west-2.compute.amazonaws.com]
changed: [ec2-13-135-103-176.eu-west-2.compute.amazonaws.com]

PLAY RECAP *****
ec2-13-135-103-176.eu-west-2.compute.amazonaws.com : ok=14  changed=9  unreachable=0  failed=0  skipped=0  rescued=0
ignored=0
ec2-18-130-100-205.eu-west-2.compute.amazonaws.com : ok=14  changed=1  unreachable=0  failed=0  skipped=0  rescued=0
ignored=0
ec2-18-134-245-218.eu-west-2.compute.amazonaws.com : ok=14  changed=1  unreachable=0  failed=0  skipped=0  rescued=0
ignored=0
ec2-18-170-39-85.eu-west-2.compute.amazonaws.com : ok=14  changed=1  unreachable=0  failed=0  skipped=0  rescued=0
ignored=0
```

All is working as expected 🥳

And if we reduce the number of servers, ansible will detect it as well. And the playbook will be executed only on those servers ✅

If we decide that the aws_ec2 inventory file will be the one that we always use then we can make it the default one., so we don't pass as argument to the terminal



```
[defaults]
host_key_checking = False
inventory = inventory_aws_ec2.yaml

enable_plugins = aws_ec2

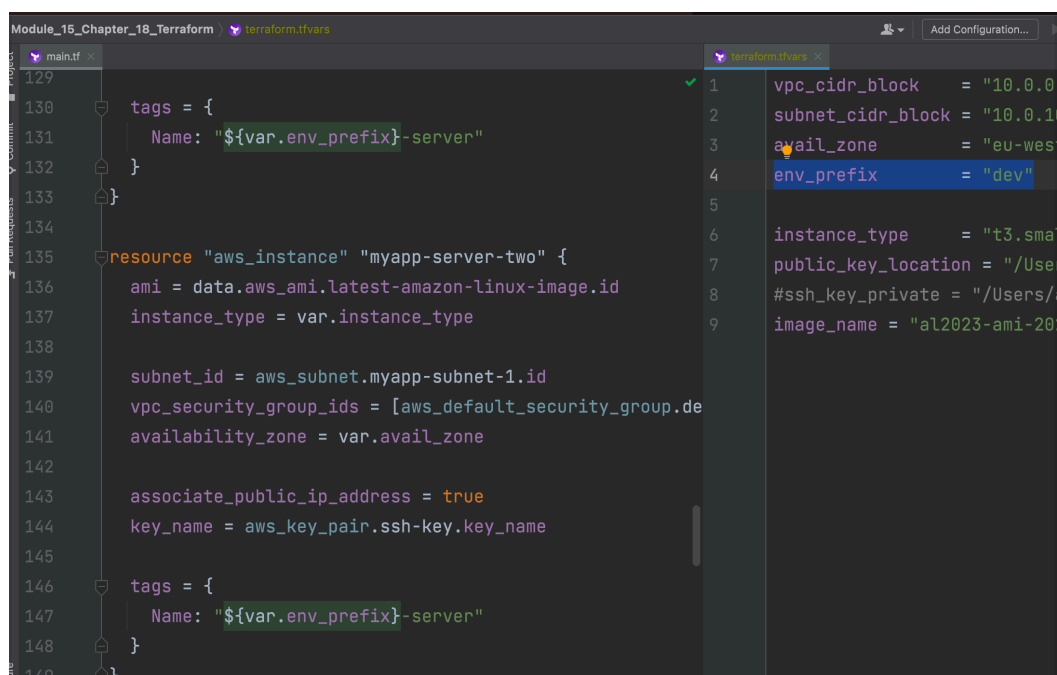
remote_user = ec2-user
private_key_file = ~/.ssh/id_ed25519
```

Target only specific servers

Now let's say we don't want to execute the playbook on all the servers in the aws region, we only want to target some of the servers.

So let's destroy the infra

and create a new one with 2 dev servers



```
main.tf
129
130 tags = {
131   Name: "${var.env_prefix}-server"
132 }
133
134
135 resource "aws_instance" "myapp-server-two" {
136   ami = data.aws_ami.latest-amazon-linux-image.id
137   instance_type = var.instance_type
138
139   subnet_id = aws_subnet.myapp-subnet-1.id
140   vpc_security_group_ids = [aws_default_security_group.default.id]
141   availability_zone = var.avail_zone
142
143   associate_public_ip_address = true
144   key_name = aws_key_pair.ssh-key.key_name
145
146   tags = {
147     Name: "${var.env_prefix}-server"
148   }
149 }

terraform.tfvars
1 vpc_cidr_block = "10.0.0.0/24"
2 subnet_cidr_block = "10.0.1.0/24"
3 avail_zone = "eu-west-1"
4 env_prefix = "dev"
5
6 instance_type = "t3.small"
7 public_key_location = "/Users/astrid/.ssh/id_rsa.pub"
8 #ssh_key_private = "/Users/astrid/.ssh/id_rsa"
9 image_name = "al2023-ami-20230815"
```

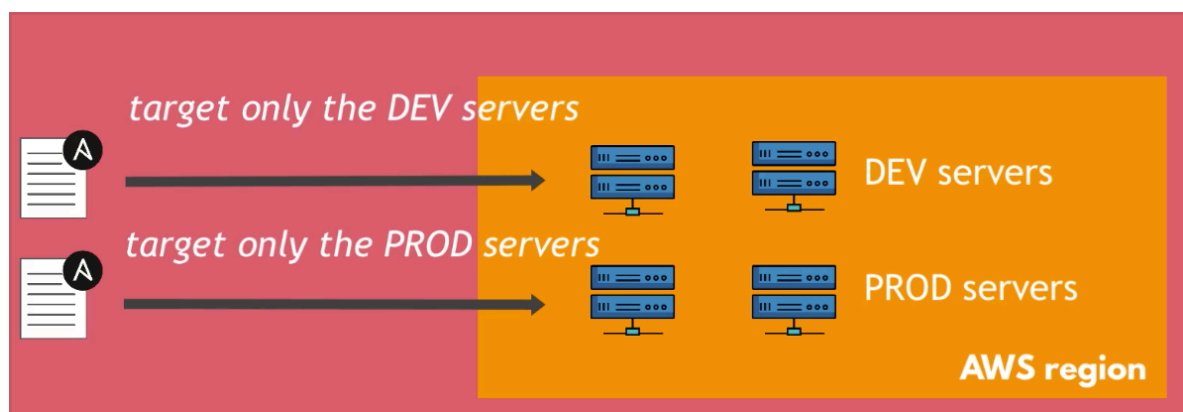
and 2 prod servers

```

160     key_name = aws_key_pair.ssh-key.key_name
161
162     tags = {
163         Name: "prod-server"
164     }
165 }
166
167 resource "aws_instance" "myapp-server-four" {
168     ami = data.aws_ami.latest-amazon-linux-image.id
169     instance_type = var.instance_type
170
171     subnet_id = aws_subnet.myapp-subnet-1.id
172     vpc_security_group_ids = [aws_default_security_group.default-sg.id]
173     availability_zone = var.avail_zone
174
175     associate_public_ip_address = true
176     key_name = aws_key_pair.ssh-key.key_name
177
178     tags = {
179         Name: "prod-server"
180     }

```

As we want the different configuration one each environment.



Apply complete! Resources: 10 added, 0 changed, 0 destroyed.

Outputs:

```

aws-ami_id = "ami-01c952cfc86b7870d"
ec2-public_ip_1 = "3.8.174.28"
ec2-public_ip_2 = "35.176.178.129"
ec2-public_ip_3 = "18.133.244.235"
ec2-public_ip_4 = "35.178.12.248"

```

~/Pyt/Module_15_Chapter_18_Terraform ➤ Module_15/An...19_Terraform !1 ?2 ➤

☐	Name	▲	Insta
☒	dev-server		i-016
☐	dev-server		i-09b
☐	prod-server		i-0c7
☐	prod-server		i-07b

So we want to filter based on the name -> the tag is 'Name'

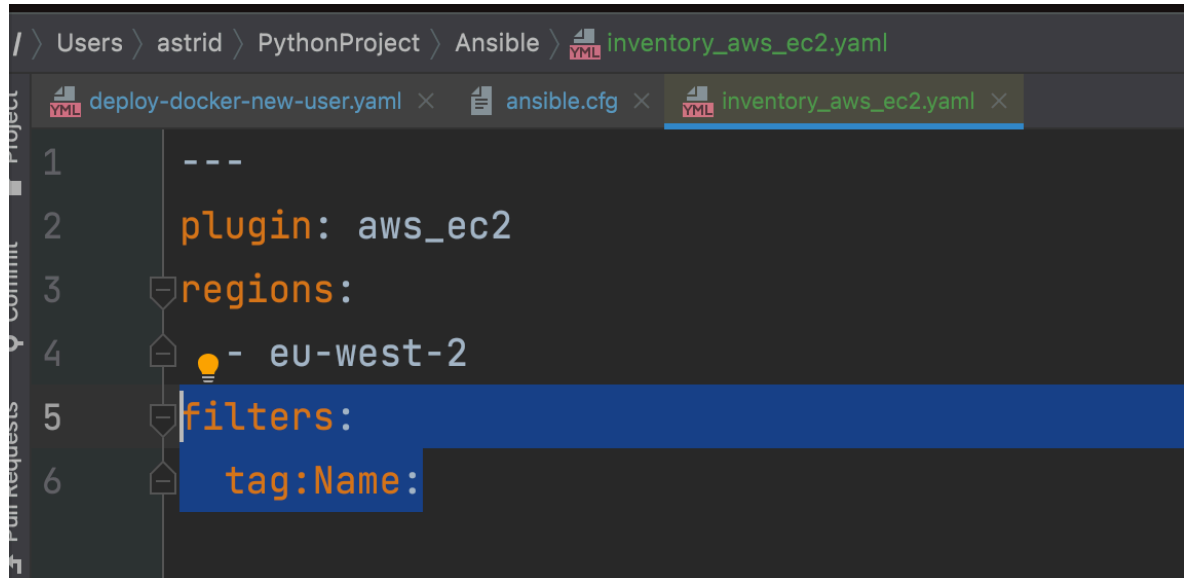
☐	Name	▲	Instance ID
☒	dev-server		i-016f781e205e79bee
☐	dev-server		i-09b9f4a9575614b2f
☐	prod-server		i-0c721ffa04032feee
☐	prod-server		i-07b0c8a8d9e1a2615

filter based on
name

And tag is an attribute. A tag is an object of type dictionary so we need the

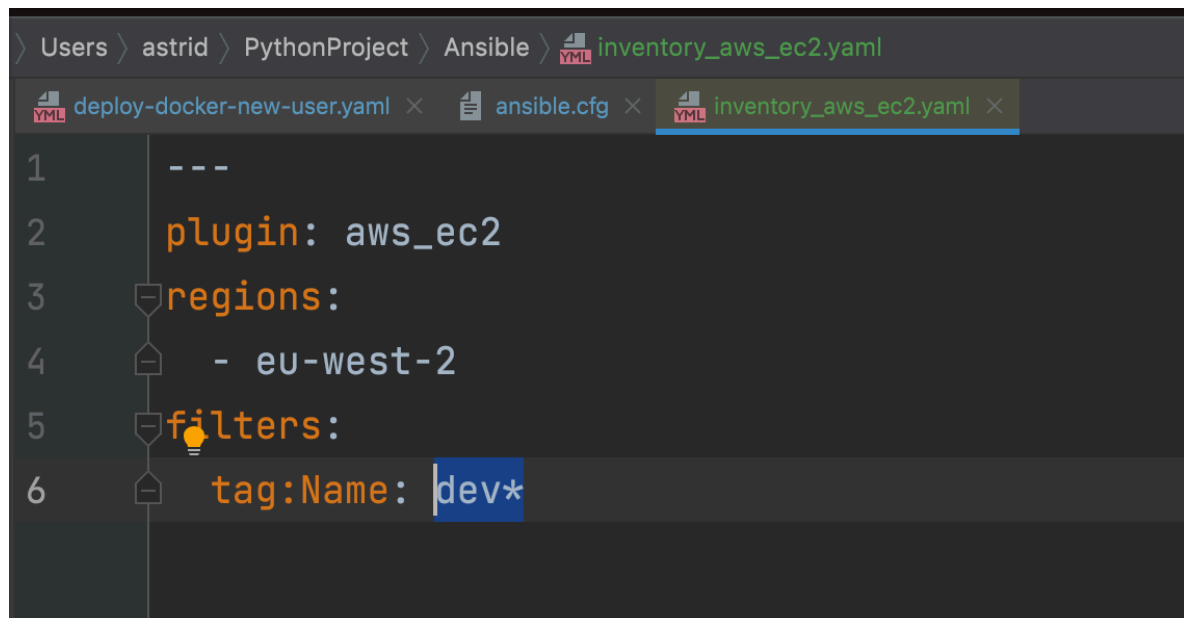
<key> after the tag which is 'Name'

- **tag:<key>** - The key/value combination of a tag assigned to the resource. Use the tag key in the filter name and the tag value as the filter value. For example, to find all resources that have a tag with the key `Owner` and the value `TeamA`, specify **tag:Owner** for the filter name and `TeamA` for the filter value.



```
1  ---
2  plugin: aws_ec2
3  regions:
4  - eu-west-2
5  filters:
6  tag:Name:
```

And then we specify the value of the 'Name' which is everything starting with 'dev' as we want to catch the servers with name dev-server



```
1  ---
2  plugin: aws_ec2
3  regions:
4  - eu-west-2
5  filters:
6  tag:Name: dev*
```

And we can easily test that we are fetching what we want using
-> ansible-inventory -i inventory_aws_ec2.yaml --graph

```

> ansible-inventory -i inventory_aws_ec2.yaml --graph
[WARNING]: Deprecation warnings can be disabled by setting `deprecation_warnings=False` in ansible.cfg.
[DEPRECATION WARNING]: The 'tags' host variable is deprecated. Use 'ec2_tags' instead. This feature will be removed in a release after 2026-12-01.
@all:
  |--@ungrouped:
  |--@aws_ec2:
    |--ec2-35-176-178-129.eu-west-2.compute.amazonaws.com
    |--ec2-3-8-174-28.eu-west-2.compute.amazonaws.com

```

And if we want only the running instances



```

1  ---
2  plugin: aws_ec2
3  regions:
4  - eu-west-2
5  filters:
6    tag:Name: dev*
7    instance-state-name: running

```

And if I test

-> ansible-inventory -i inventory_aws_ec2.yaml --graph

```

> ansible-inventory -i inventory_aws_ec2.yaml --graph
[WARNING]: Deprecation warnings can be disabled by setting `deprecation_warnings=False` in ansible.cfg.
[DEPRECATION WARNING]: The 'tags' host variable is deprecated. Use 'ec2_tags' instead. This feature will be removed in a release after 2026-12-01.
@all:
  |--@ungrouped:
  |--@aws_ec2:
    |--ec2-35-176-178-129.eu-west-2.compute.amazonaws.com
    |--ec2-3-8-174-28.eu-west-2.compute.amazonaws.com

```

And if I want to see more details of the inventory I use '--list'

-> ansible-inventory -i inventory_aws_ec2.yaml --list

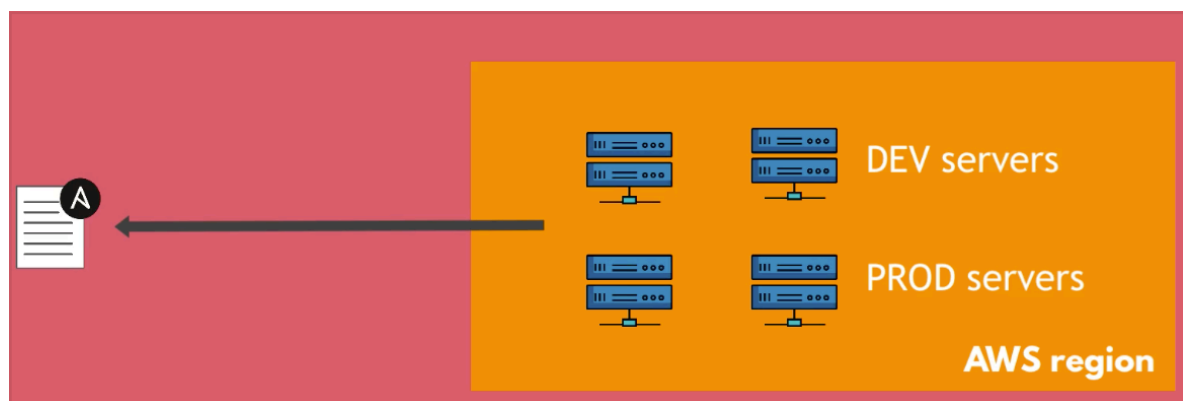
```

        "threads_per_core": 2
    },
    "current_instance_boot_mode": {
        "__ansible_unsafe": "uefi"
    },
    "ebs_optimized": false,
    "ec2_tags": {
        "Name": {
            "__ansible_unsafe": "dev-server"
        }
    },
    "ena_support": true,
    "enclave_options": {
        "enabled": false
    },

```

Create dynamic groups

In the case we want to return both dev and prod servers with the ansible inventory plugin but we want to differentiate them.

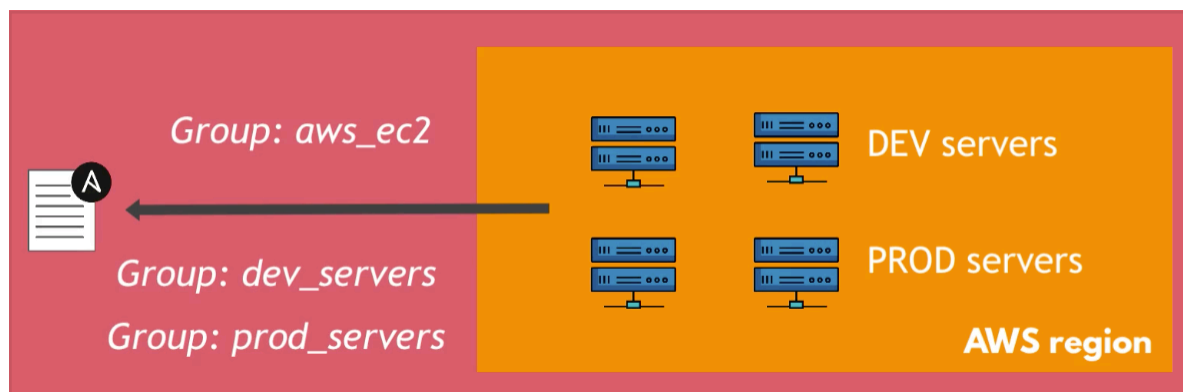


We want to group the servers into their separate groups.

Instead of returning all of the servers and having them in 'aws_ec2' group

```
@all:
  |--@ungrouped:
  |--@aws_ec2:
    |--ec2-35-176-178-129.eu-west-2.compute.amazonaws.com
    |--ec2-3-8-174-28.eu-west-2.compute.amazonaws.com
```

We want one inventory group for dev servers and another inventory group for prod.



For that we use another configuration attribute called 'keyed_groups'

keyed_groups list / elements=dictionary	Add hosts to group based on the values of a variable. Default: []
default_value string added in ansible-core 2.12	The default value when the host variable's value is <code>None</code> or an empty string. This option is mutually exclusive with <code>keyed_groups[].trailing_separator</code> .
key string	The key from input dictionary used to generate groups.
parent_group string	parent group for <code>keyed</code> group.
prefix string	A <code>keyed</code> group name will start with this prefix. Default: ""
separator string	separator used to build the <code>keyed</code> group name. Default: "-"
trailing_separator boolean added in ansible-core 2.12	Set this option to <code>false</code> to omit the <code>keyed_groups[].separator</code> after the host variable when the value is <code>None</code> or an empty string. This option is mutually exclusive with <code>keyed_groups[].default_value</code> . Choices: <code>false</code> <code>true</code> ← (default)
leading_separator boolean added in ansible-core 2.11	Use in conjunction with <code>keyed_groups</code> . By default, a <code>keyed</code> group that does not have a prefix or a separator provided will have a name that starts with an underscore. This is because the default prefix is "" and the default separator is "-".

And we will use 'key' configuration from 'keyed_groups'. With 'key' we need to define what is the information of the instances that we want to use for grouping. And in this demo we want to use 'tags' to group them, see the value from the list returned by

-> ansible-inventory -i inventory_aws_ec2.yaml --list

```

        "__ansible_unsafe": "subnet-01aeba7edd2642c16"
    },
    "tags": {
        "Name": {
            "__ansible_unsafe": "dev-server"
        }
    }
},

```

Note that for 'filters' we use attributes from the list provided in the documentation but for 'keyed_groups' we use the ones provided by

-> `ansible-inventory -i inventory_aws_ec2.yaml --list`

Which come directly from the instances.

```

> Users > astrid > PythonProject > Ansible > inventory_aws_ec2.yaml
YML deploy-docker-new-user.yaml x ansible.cfg x YML inventory_aws_ec2.yaml x
1 plugin: aws_ec2
2
3 regions:
4   - eu-west-2
5   #filters:
6   #  tag:Name: dev*
7   #  instance-state-name: running
8   keyed_groups:
9     - key: tags

```

Each server only have one tag and is the tag 'Name' and now we execute the plugin and let's see what we get

-> `ansible-inventory -i inventory_aws_ec2.yaml --graph`

```

) ansible-inventory -i inventory_aws_ec2.yaml --graph
[WARNING]: Deprecation warnings can be disabled by setting `deprecation_warnings=False` in ansible.cfg
[DEPRECATION WARNING]: The 'tags' host variable is deprecated. Use 'ec2_tags' instead. This message will be
shown on a release after 2026-12-01.
@all:
  |--@ungrouped:
  |--@aws_ec2:
    |--ec2-35-178-12-248.eu-west-2.compute.amazonaws.com
    |--ec2-18-133-244-235.eu-west-2.compute.amazonaws.com
    |--ec2-35-176-178-129.eu-west-2.compute.amazonaws.com
    |--ec2-3-8-174-28.eu-west-2.compute.amazonaws.com
  |--@_Name_prod_server:
    |--ec2-35-178-12-248.eu-west-2.compute.amazonaws.com
    |--ec2-18-133-244-235.eu-west-2.compute.amazonaws.com
  |--@_Name_dev_server:
    |--ec2-35-176-178-129.eu-west-2.compute.amazonaws.com
    |--ec2-3-8-174-28.eu-west-2.compute.amazonaws.com

```

We get 3 groups, one with all the servers in one group and two more groups one for the dev servers and another for the prod servers.

The groups that differentiate dev from prod have a name with a prefix which is an underscore '_'

```

  |--@_Name_prod_server:
    |--ec2-35-178-12-248
    |--ec2-18-133-244-23
  |--@_Name_dev_server:
    |--ec2-35-176-178-12

```

If we don't want that we can have our own prefix

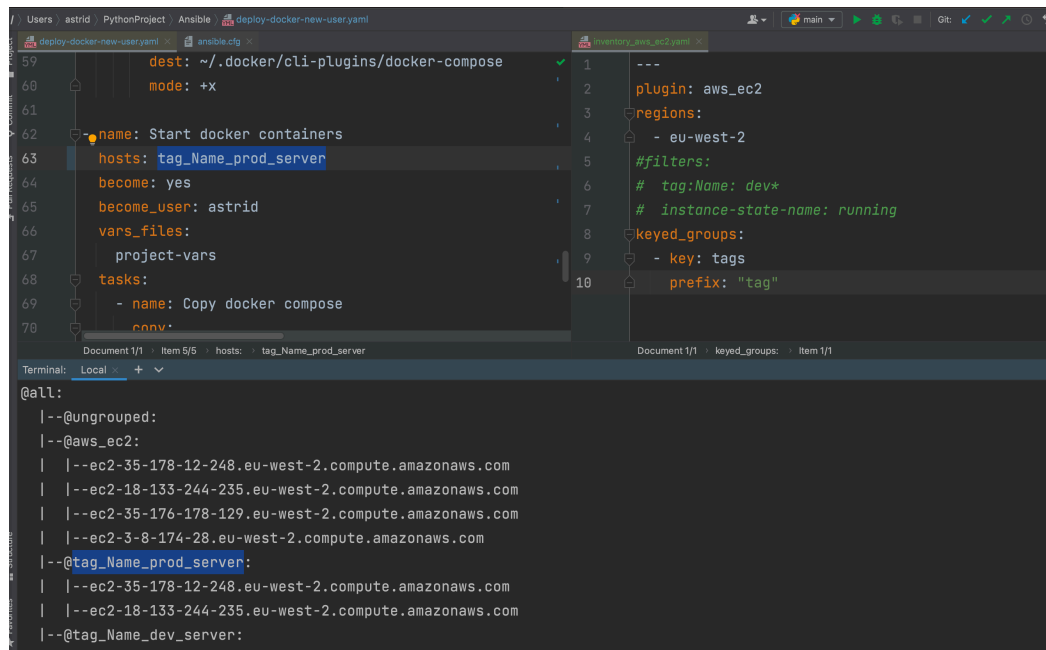
```
> Users > astrid > PythonProject > Ansible > inventory_aws_ec2.yaml
YML deploy-docker-new-user.yaml x ansible.cfg x YML inventory_aws_ec2.yaml x
1 ---
2 plugin: aws_ec2
3 regions:
4   - eu-west-2
5   #filters:
6   #  tag:Name: dev*
7   #  instance-state-name: running
8 keyed_groups:
9   - key: tags
10  prefix: "tag"
```

And we run it

-> ansible-inventory -i inventory_aws_ec2.yaml --graph

```
@all:
  |--@ungrouped:
  |--@aws_ec2:
  | |--ec2-35-178-12-248.eu-west-2.compute.amazonaws.com
  | |--ec2-18-133-244-235.eu-west-2.compute.amazonaws.com
  | |--ec2-35-176-178-129.eu-west-2.compute.amazonaws.com
  | |--ec2-3-8-174-28.eu-west-2.compute.amazonaws.com
  |--@tag_Name_prod_server:
  | |--ec2-35-178-12-248.eu-west-2.compute.amazonaws.com
  | |--ec2-18-133-244-235.eu-west-2.compute.amazonaws.com
  |--@tag_Name_dev_server:
  | |--ec2-35-176-178-129.eu-west-2.compute.amazonaws.com
  | |--ec2-3-8-174-28.eu-west-2.compute.amazonaws.com
```

So now that I have the groups name and I want to configure only the prod servers then I can update the hosts value in the playbook



The screenshot shows a code editor with two files: `deploy-docker-new-user.yml` and `inventory_aws_ec2.yml`. The first file is an Ansible playbook with tasks to copy a Docker compose file and start containers. The second file is an inventory listing AWS EC2 instances grouped by region and tagged by name. Below the editor, a terminal window shows the output of running the playbook, displaying the list of hosts for each group.

```
59 dest: ~/.docker/cli-plugins/docker-compose
60 mode: +x
61
62 - name: Start docker containers
63   hosts: tag_Name_prod_server
64   become: yes
65   become_user: astrid
66   vars_files:
67     - project-vars
68   tasks:
69     - name: Copy docker compose
70       copy:
71         src: docker-compose.yml
72         dest: ~/.docker/cli-plugins/docker-compose
73         mode: +x
```

```
1 ---
2 plugin: aws_ec2
3 regions:
4   - eu-west-2
5 #filters:
6 # tag:Name: dev*
7 # instance-state-name: running
8 keyed_groups:
9   - key: tags
10    prefix: "tag"
```

```
@all:
 |--@ungrouped:
 |--@aws_ec2:
 | |--ec2-35-178-12-248.eu-west-2.compute.amazonaws.com
 | |--ec2-18-133-244-235.eu-west-2.compute.amazonaws.com
 | |--ec2-35-176-178-129.eu-west-2.compute.amazonaws.com
 | |--ec2-3-8-174-28.eu-west-2.compute.amazonaws.com
 |--@tag_Name_prod_server:
 | |--ec2-35-178-12-248.eu-west-2.compute.amazonaws.com
 | |--ec2-18-133-244-235.eu-west-2.compute.amazonaws.com
 |--@tag_Name_dev_server:
```

Now if I execute the playbook
-> `ansible-playbook deploy-docker-new-user.yml`



The screenshot shows the output of the Ansible command. It displays the results for the 'Docker login' and 'Start containers from compose' tasks, followed by a 'PLAY RECAP' summary. The summary shows that 14 hosts were changed, 9 of which were in the 'tag_Name_prod_server' group, and 1 host in the 'tag_Name_dev_server' group.

```
TASK [Docker login] *****
changed: [ec2-18-133-244-235.eu-west-2.compute.amazonaws.com]
changed: [ec2-35-178-12-248.eu-west-2.compute.amazonaws.com]

TASK [Start containers from compose] *****
changed: [ec2-18-133-244-235.eu-west-2.compute.amazonaws.com]
changed: [ec2-35-178-12-248.eu-west-2.compute.amazonaws.com]

PLAY RECAP *****
ec2-18-133-244-235.eu-west-2.compute.amazonaws.com : ok=14  changed=9  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0
ec2-3-8-174-28.eu-west-2.compute.amazonaws.com : ok=1  changed=0  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0
ec2-35-176-178-129.eu-west-2.compute.amazonaws.com : ok=1  changed=0  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0
ec2-35-178-12-248.eu-west-2.compute.amazonaws.com : ok=14  changed=9  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0
```

And only the prod servers got configured 🤔

If we want to also group the servers based on the instance type (Nana has some servers as t2.micro and others as t2.small in my case all my servers have the same type)

Nana executes the list with the servers info to figure out the attribute for the instance type

-> `ansible-inventory -i inventory_aws_ec2.yml --list`

```
ansible.cfg M  ! deploy-docker-new-user.yaml U  ! inventory_aws_ec2.yaml U ●
1 ---
2 plugin: aws_ec2
3 regions:
4   - eu-central-1
5 keyed_groups:
6   - key: tags
7     prefix: "tag"
8   -
9
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

```

    "hibernation_options": {
      "configured": false
    },
    "hypervisor": "xen",
    "image_id": "ami-03cceb19496c25679",
    "instance_id": "i-07b0c8a8d9e1a2615",
    "instance_type": "t2.small",
    "key_name": "server-key",
    "launch_time": "2024-02-16T11:46:11+00:00",
    "maintenance_options": {
      "auto_recovery": "default"
    },
    "metadata_options": {
```

And use that in the inventory plugin yaml file as a key

```
ansible.cfg M  ! deploy-docker-new-user.yaml U  ! inventory_aws_ec2.yaml U ×
1 ---
2 plugin: aws_ec2
3 regions:
4   - eu-central-1
5 keyed_groups:
6   - key: tags
7     prefix: tag
8   - key: instance_type
9     prefix: instance_type
10
```

And this is the result

```
nana@macbook /Users/nana/ansible [master]
● % ansible-inventory -i inventory_aws_ec2.yaml --graph
@all:
|--@ungrouped:
|--aws_ec2:
|   |--ec2-18-153-82-192.eu-central-1.compute.amazonaws.com
|   |--ec2-52-58-250-26.eu-central-1.compute.amazonaws.com
|   |--ec2-3-122-112-60.eu-central-1.compute.amazonaws.com
|   |--ec2-3-66-88-103.eu-central-1.compute.amazonaws.com
|--@tag_Name_dev_server:
|   |--ec2-18-153-82-192.eu-central-1.compute.amazonaws.com
|   |--ec2-3-122-112-60.eu-central-1.compute.amazonaws.com
|--@instance_type_t2_micro:
|   |--ec2-18-153-82-192.eu-central-1.compute.amazonaws.com
|   |--ec2-3-122-112-60.eu-central-1.compute.amazonaws.com
|--@tag_Name_prod_server:
|   |--ec2-52-58-250-26.eu-central-1.compute.amazonaws.com
|   |--ec2-3-66-88-103.eu-central-1.compute.amazonaws.com
|--@instance_type_t2_small:
|   |--ec2-52-58-250-26.eu-central-1.compute.amazonaws.com
|   |--ec2-3-66-88-103.eu-central-1.compute.amazonaws.com
nana@macbook /Users/nana/ansible [master]
```